НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
Факультет программной инженерии и компьютерных технологий

Тестирование программного обеспечения

Лабораторная работа № 1

Выполнили студенты:
Сидимеков Дмитрий Алексеевич
Казанцева Елизавета Константиновна
Группа № P3322
Преподаватель:
Демурчян Владимир Варламович

г. Санкт-Петербург
2026

Оглавление

Задание: 3

Выполнение:..... 4

Код: 4

Задание 1:..... 4

Задание 2:..... 6

Задание 3:..... 8

Вывод:..... 14

Задание:

1. Для указанной функции провести модульное тестирование разложения функции в степенной ряд. Выбрать достаточное тестовое покрытие.
2. Провести модульное тестирование указанного алгоритма. Для этого выбрать характерные точки внутри алгоритма, и для предложенных самостоятельно наборов исходных данных записать последовательность попадания в характерные точки. Сравнить последовательность попадания с эталонной.
3. Сформировать доменную модель для заданного текста. Разработать тестовое покрытие для данной доменной модели

Вариант: 346230

Введите вариант: 346230

1. Функция cos(x)
2. Программный модуль для сортировки массива методом пузырька (http://www.cs.usfca.edu/~galles/visualization/ComparisonSort.html)
3. Описание предметной области:

> Простетный Вогон Джельц улыбнулся очень медленно. Он сделал так не ради эффекта, а потому что не мог вспомнить правильную последовательность движения мышц. Он только что побаловал себя освежающей серией воплей на своих пленников, и теперь чувствовал себя отдохнувшим и готовым к небольшой гнусности.

Выполнение

Код:

https://github.com/sidimekov/itmo-tpo-lab1

Задание 1

Функция: cos(x)

Область определения: $\boldsymbol{x} \in \mathbb{R}$, область значений: $[-1; 1]$ (при достаточных n - частичные суммы ряда могут выйти за отрезок).

Зоны анализа эквивалентности:

- Монотонное убывание на $[0; \pi/2]$ при y > 0
- Монотонное убывание на $[\pi/2; \pi]$ при y < 0
- Монотонное возрастание на $[\pi; 3\pi/2]$ при y < 0
- Монотонное возрастание на $[3\pi/2; 2\pi]$ при y > 0
- Максимумы: $\{2\pi n,\ n \in \mathbb{Z}\}$, минимумы: $\{\pi + 2\pi n,\ n \in \mathbb{Z}\}$
- Точки перемены знака с + на - $\{\pi/2 + 2\pi n,\ n \in \mathbb{Z}\}$
- Точки перемены знака с - на + $\{3\pi/2 + 2\pi n,\ n \in \mathbb{Z}\}$

Из свойств периодичности 2π зоны повторяются, поэтому достаточно протестировать по точке из каждой зоны и само свойство периодичности (при достаточно больших n для точности косинуса), а также свойство чётности функции

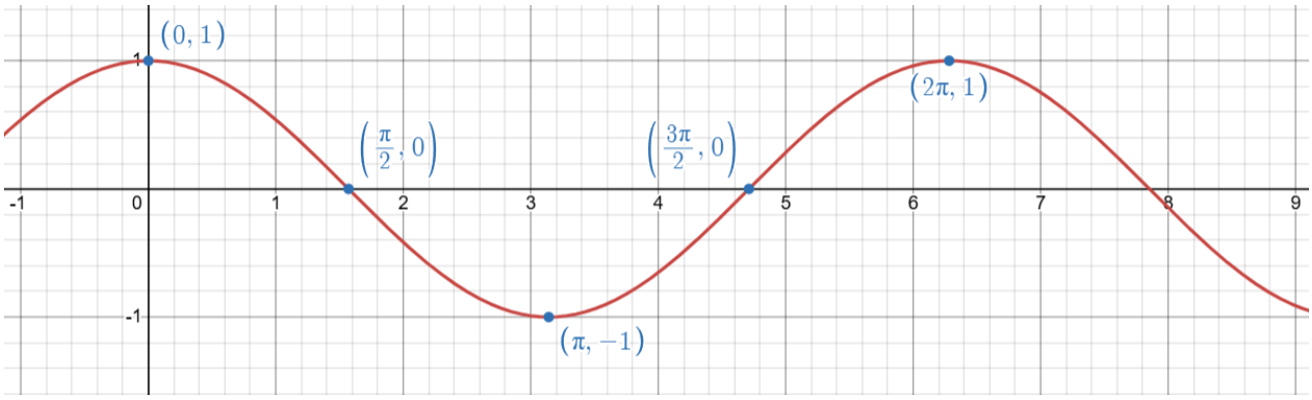


Рисунок 1. График cos(x) и табличные значения

Тестовое покрытие

1. Базовые точки

X	0.0	1.0	1.57	2.0	3.14	4.0	4.71	5.0	6.28	7.0
Cos x	1.0	0.54	0.00	-0.42	-1.0	-0.65	0.0	0.28	1.00	0.75

Для маленьких n сравниваем функцию с частичными предпосчитанными суммами.
Например:

- при x = 1, n = 1 ожидаем 1
- при x = 1, n = 2 ожидаем 1 - 1/2
- при x = 1, n = 3 ожидаем 1 - 1/2 + 1/24

Возьмём n: 1,2,3,5
Для больших n будем сравнивать сумму с какой-то дельтой с самой функцией cos(x).

2. Проверка свойств (property-based тесты)

- Чётность: cos(x) = cos(-x)
- Периодичность: на больших n проверять что оба значения с периодом 2pi с одинаковым значением cos(x) достаточно близки

3. Проверка больших / малых и предельных значений

Проверка, что реализация ведёт себя корректно на больших/малых числах, и числах, близких к границам типа double.

- при очень малых x (взяли 1e-8, 1e-12) функция cos(x) близка к 1, а члены ряда быстро убывают, поэтому вычисление должно оставаться конечным и корректным.
- при x = ±100 значения уже большие для разложения около нуля, однако они ещё не приводят к немедленному переполнению, поэтому тест проверяет, что метод хотя бы возвращает конечное число, а не NaN и не Infinity.

- Возьмём также предельные значения Double.MAX_VALUE, Double.MIN_VALUE. Проверяем, что программа предсказуемо ведёт себя на экстремальном вводе.
  Даже при небольшом числе членов ряда, например при n = 2, уже возникает вычисление x^2, которое для Double.MAX_VALUE выходит за границы представимого диапазона и приводит к переполнению. В результате метод должен возвращать Infinity или NaN.
  Double.MIN_VALUE - минимальное положительное число больше нуля. С математической точки зрения cos от этой точки должен быть близким к 1.

4. Граничные случаи

Случаи n = 0 и n < 0 являются граничными и некорректными, поэтому должны обрабатываться отдельно. Предусмотрено выбрасывание исключения IllegalArgumentException

com.github.sidimekov	85% (6/7)	95% (19/20)	90% (48/53)	95% (21/22)
task1	100% (1...	100% (2/2)	100% (11/11)	90% (9/10)
CosSeries	100% (1...	100% (2/2)	100% (11/11)	90% (9/10)

Рисунок 2. Тестовое покрытие функции разложения в степенной ряд cos(x)

Не вызывался private метод factorial(x) для x < 0, т. к. для разложения в степенной ряд используется факториал с натуральными числами.

5. Infinity, NaN

При специальных значениях Double должен возвращаться NaN.

Задание 2

Сортировка массива методом пузырька

Эталонная реализация по варианту:

http://www.cs.usfca.edu/~galles/visualization/ComparisonSort.html

Сортировка пузырьком - это простой алгоритм сортировки, который многократно проходит по массиву, сравнивая соседние элементы и меняя их местами, если они находятся в неправильном порядке. Проходы по массиву повторяются до тех пор, пока массив не будет отсортирован.

Для обеспечения корректности работы алгоритма необходимо тщательно протестировать его на различных входных данных:

- Сортировка уже отсортированного массива.
- Сортировка массива, отсортированного в обратном порядке.
- Сортировка массива со случайным порядком элементов.
- Сортировка граничных случаев (пустой массив, массив с одним элементом).
- Сортировка массивов с дубликатами, отрицательными числами и граничными значениями (Integer.MIN_VALUE, Integer.MAX_VALUE).

Цель тестирования - убедиться, что алгоритм корректно сортирует любые входные данные и не содержит ошибок (например, выхода за границы массива).

Будем проверять нашу реализацию структуры данных следующим образом:

- для публичного метода sort попытаемся выделить основные сценарии работы метода (то есть возможные ветвления для метода);
- проверять будем как "черный ящик" — подавая на вход различные наборы данных и сравнивая результат с ожидаемым, отсортированным массивом.

В ходе тестирования все тесты успешно проходятся:

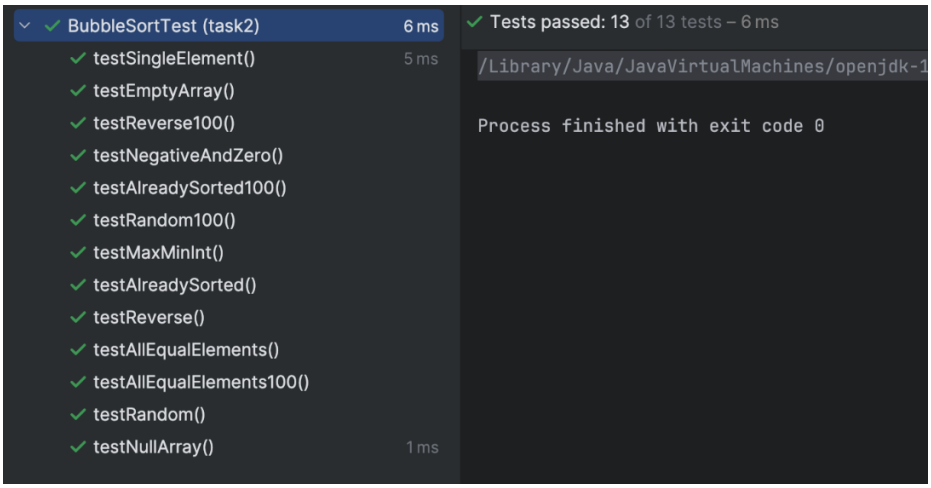
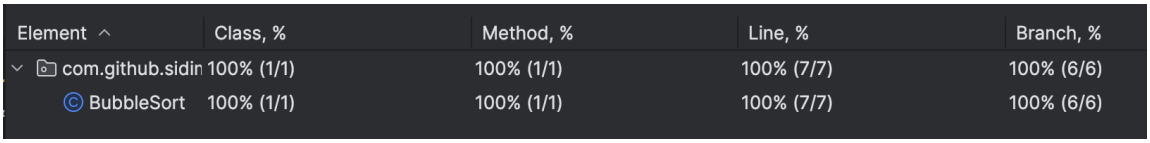


Рисунок 3. Тесты для сортировки пузырьком

Итоговое тестовое покрытие методов класса **BubbleSort**:



Element	Class, %	Method, %	Line, %	Branch, %
com.github.sidin	100% (1/1)	100% (1/1)	100% (7/7)	100% (6/6)
BubbleSort	100% (1/1)	100% (1/1)	100% (7/7)	100% (6/6)

*Рисунок 4. Тестовое покрытие методов класса **BubbleSort***

Результаты тестирования:

- Алгоритм сортировки пузырьком реализован корректно и проходит все тесты.
- Код покрыт модульными тестами, включая граничные случаи и классы эквивалентности.
- Тестирование подтвердило, что сортировка корректно обрабатывает любые входные массивы целых чисел.

Задание 3

Текст: *"Простетный Вогон Джельц улыбнулся очень медленно. Он сделал так не ради эффекта, а потому что не мог вспомнить правильную последовательность движения мышц. Он только что побаловал себя*

освежающей серией воплей на своих пленников, и теперь чувствовал себя отдохнувшим и готовым к небольшой гнусности."

Спроектированная UML диаграмма предметной области.

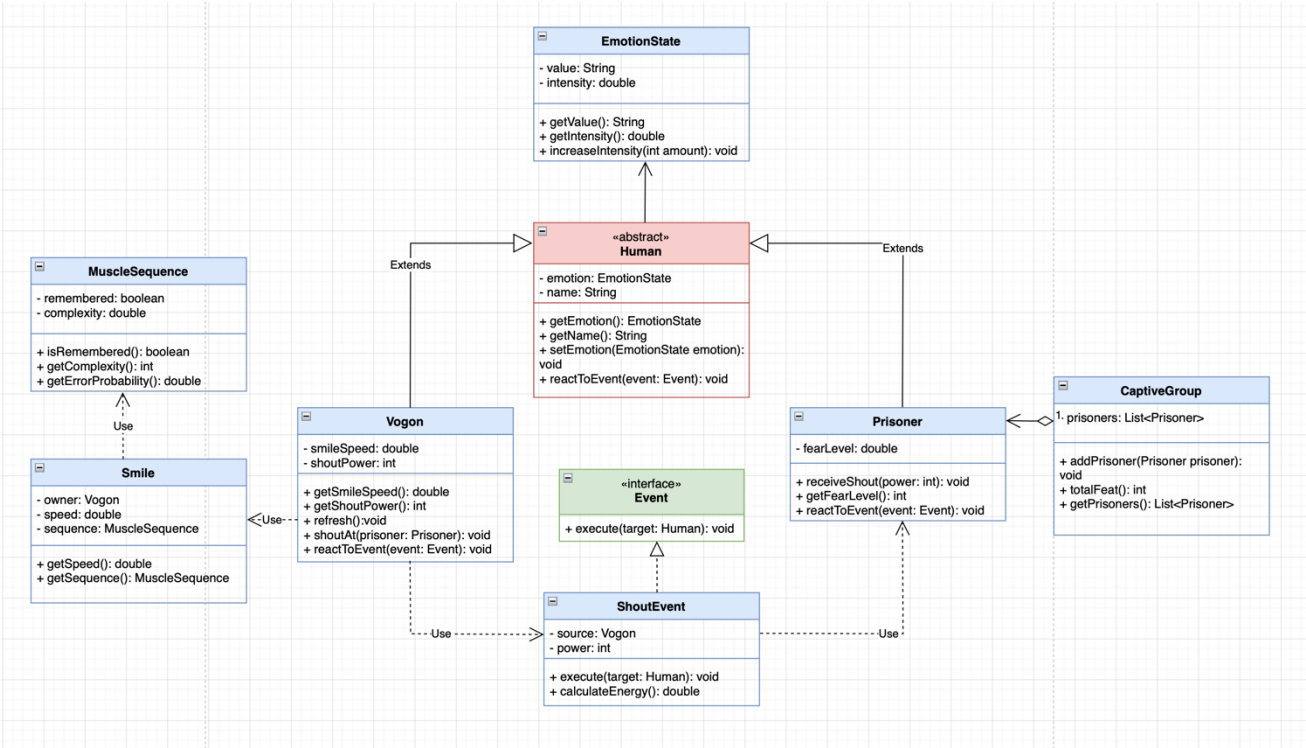


Рисунок 5. UML диаграмма предметной области

Предметная область включает в себя:

Персонажи

- Вогон Джельц — класс **Vogon.**
- Пленники — класс **Prisoner.**

Свойства

Состояние персонажей описывается следующими параметрами:

- эмоциональное состояние (**EmotionState**)
    - значение эмоции (**value**)
    - интенсивность (**intensity**)
- скорость улыбки (**smileSpeed**)
- сила крика (**shoutPower**)
- уровень страха (**fearLevel**)

Действия и события

В системе реализованы следующие действия:

- улыбнуться

  реализуется через вычисление времени улыбки на основе последовательности мышц (**MuscleSequence**):

$$time = complexity \,/\, smileSpeed$$

- кричать

  реализуется через событие **ShoutEvent**

$$energy = power^2$$

$$fearIncrease = power^2 \,/\, 10$$

- воздействие на пленника

  при выполнении **ShoutEvent** у пленника увеличивается уровень страха.

- отдохнуть
  реализуется методом **refresh()** у класса **Vogon**, который изменяет эмоциональное состояние.

Далее, напишем тесты по спроектированной модели.

Цель тестирования — убедиться, что:

- все сущности системы создаются корректно;
- математические зависимости (уровень страха, энергия крика, время улыбки) вычисляются правильно;
- состояния персонажей изменяются в соответствии с выполняемыми событиями;
- события выполняются предсказуемо и не вызывают ошибок.

Тестовый сценарий:

№	Начальное состояние	Ввод	Действие системы	Итоговое состояние
1	Система не содержит объектов	Создание Вогона	Создаётся объект Vogon (name = "Prostetnic Vogon Jeltz", emotion = ("angry", 1), smileSpeed = 0.1, shoutPower = 3)	В системе существует Простетник Вогон Джельц
2	В системе существует Вогон	Создание пленника Arthur	Создаётся объект Prisoner (name = "Arthur", emotion = ("neutral", 0))	В системе существует пленник Arthur
3	В системе существует Arthur	Создание пленника Ford	Создаётся объект Prisoner (name = "Ford", emotion = ("neutral", 0))	В системе существует второй пленник Ford

4	В системе есть Вогон и пленники	Вогон начинает кричать	Создаётся ShoutEvent (source = Vogon, power = 3)	Система готова выполнить событие крика
5	Arthur имеет эмоцию neutral	Вогон кричит на Arthur	Выполняется event.execute(Arthur) → вызывается receiveShout(3)	У Arthur fearLevel = 0.9 (по формуле $3^2/10$), интенсивность эмоции увеличивается
6	Ford имеет эмоцию neutral	Вогон кричит на Ford	Выполняется event.execute(Ford)	У Ford fearLevel = 0.9, эмоция усиливается
7	Пленники испуганы	Вогон продолжает серию криков	ShoutEvent выполняется повторно для Arthur и Ford	У обоих пленников увеличивается fearLevel по формуле fearLevel += $power^2/10$
8	Вогон завершил серию воплей	Вогон освежается	Вызывается метод refresh()	Эмоциональное состояние Вогона становится "refreshed"
9	Вогон чувствует себя отдохнувшим	Вогон начинает улыбаться	Вызывается calculateSmileTime(sequence)	Время улыбки вычисляется как complexity / smileSpeed, улыбка происходит медленно

10	Система содержит Вогона и пленников	Проверка состояния персонажей	Выполняются getEmotion() и getFearLevel()	Пленники имеют повышенный fearLevel, Вогон находится в состоянии "refreshed"

Как мы можем видеть, все работает корректно:

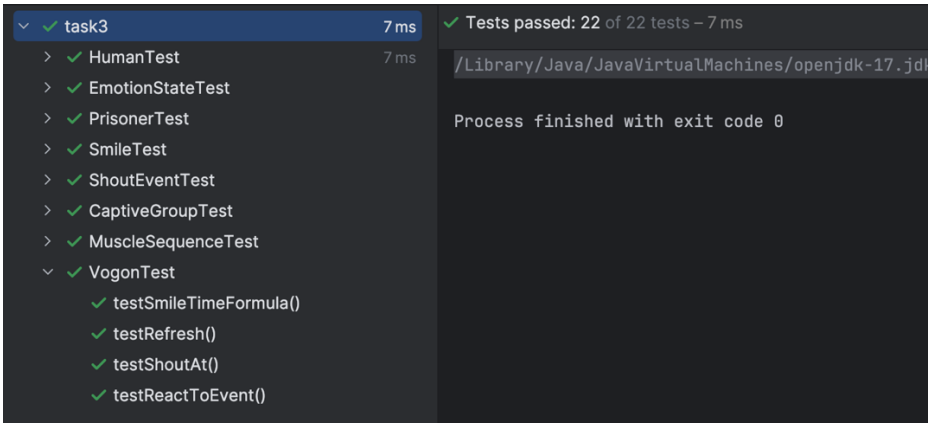
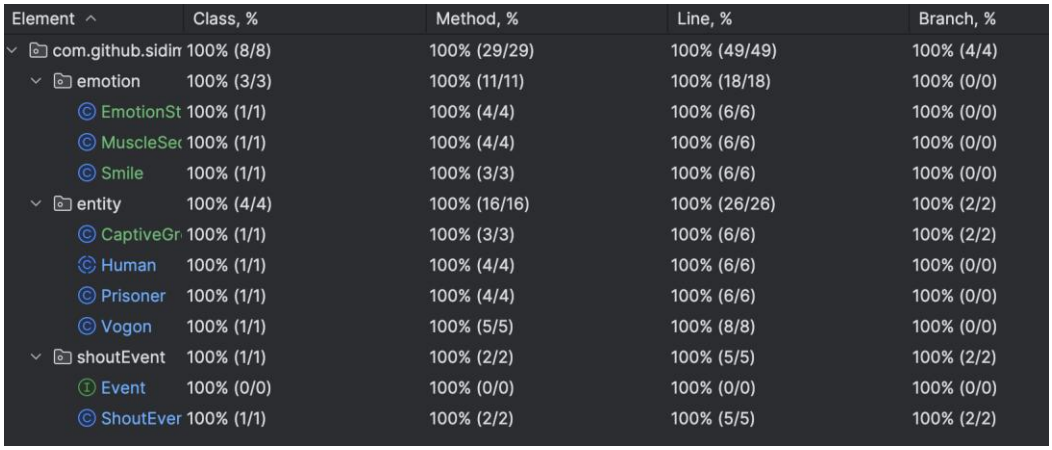


Рисунок 6. Тесты для UML диаграммы

Итоговое тестовое покрытие:



Element	Class, %	Method, %	Line, %	Branch, %
com.github.sidin	100% (8/8)	100% (29/29)	100% (49/49)	100% (4/4)
emotion	100% (3/3)	100% (11/11)	100% (18/18)	100% (0/0)
EmotionSt	100% (1/1)	100% (4/4)	100% (6/6)	100% (0/0)
MuscleSe	100% (1/1)	100% (4/4)	100% (6/6)	100% (0/0)
Smile	100% (1/1)	100% (3/3)	100% (6/6)	100% (0/0)
entity	100% (4/4)	100% (16/16)	100% (26/26)	100% (2/2)
CaptiveGr	100% (1/1)	100% (3/3)	100% (6/6)	100% (2/2)
Human	100% (1/1)	100% (4/4)	100% (6/6)	100% (0/0)
Prisoner	100% (1/1)	100% (4/4)	100% (6/6)	100% (0/0)
Vogon	100% (1/1)	100% (5/5)	100% (8/8)	100% (0/0)
shoutEvent	100% (1/1)	100% (2/2)	100% (5/5)	100% (2/2)
Event	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
ShoutEver	100% (1/1)	100% (2/2)	100% (5/5)	100% (2/2)

Рисунок 7. Тестовое покрытие предметной области

Вывод:

Во время выполнения лабораторной работы мы углубили свои знания в JUnit5, научились писать юнит-тесты, использовать параметризированные тесты и тесты на проверку составленной объектной модели.